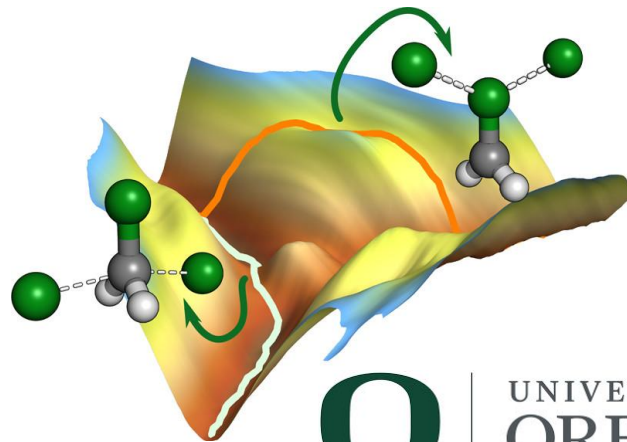
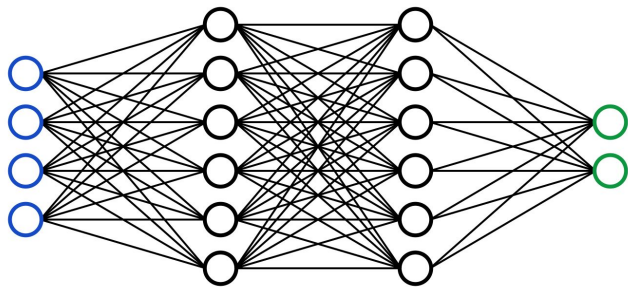


Machine Learning for Chemistry

Winter 2025

Topic 5: Decision Tree, Random Forest, Boosting



UNIVERSITY OF
OREGON

Instructor: Dr. Dhiman Ray

Assistant Professor, Department of Chemistry and Biochemistry

Overview of Topics

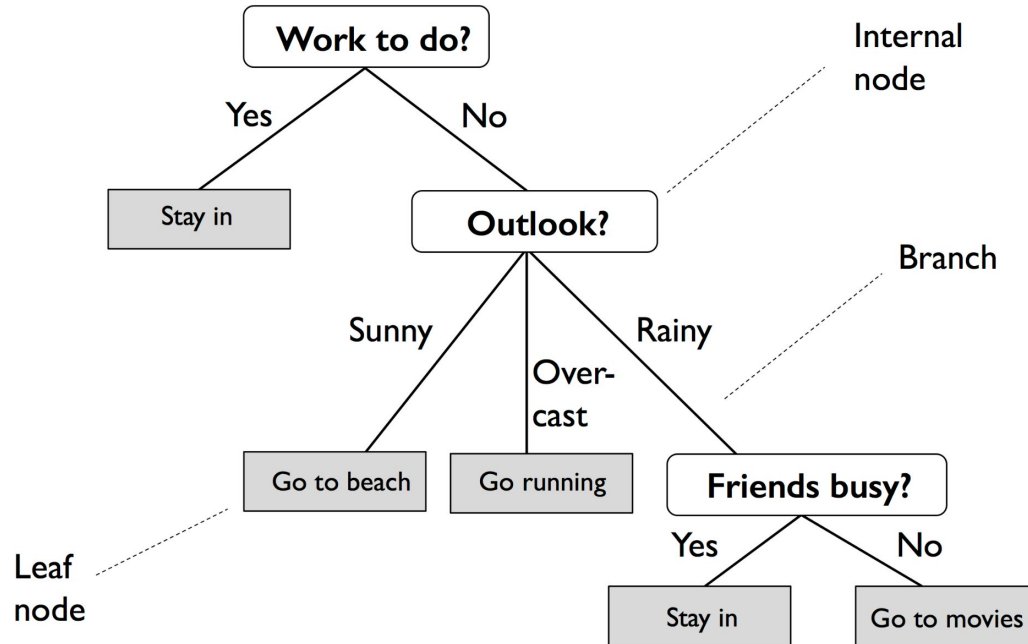
- Decision Trees
- Ensemble Learning
- Random Forest
- Gradient Boosting

References:

1. **Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow** by Aurélien Géron (Chapter 6 & 7)
2. **Machine Learning with Pytorch and Scikit-Learn** by Raschka, Liu, and Mirjalili (Chapter 3)

Decision Trees

As the name **decision tree** suggests, we can think of this model as breaking down our data by making a decision based on asking a series of questions.



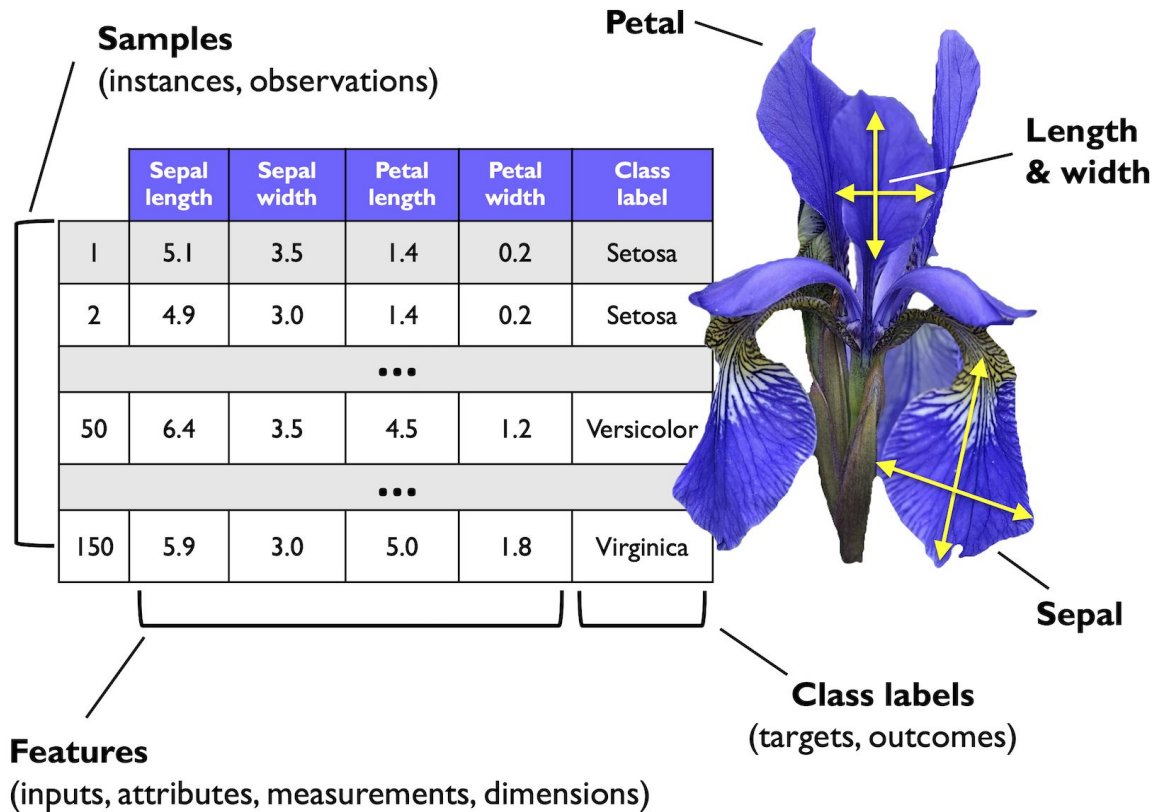
Decision Trees

Based on the features in our training dataset, the decision tree model learns a series of questions to infer the class labels of the examples.

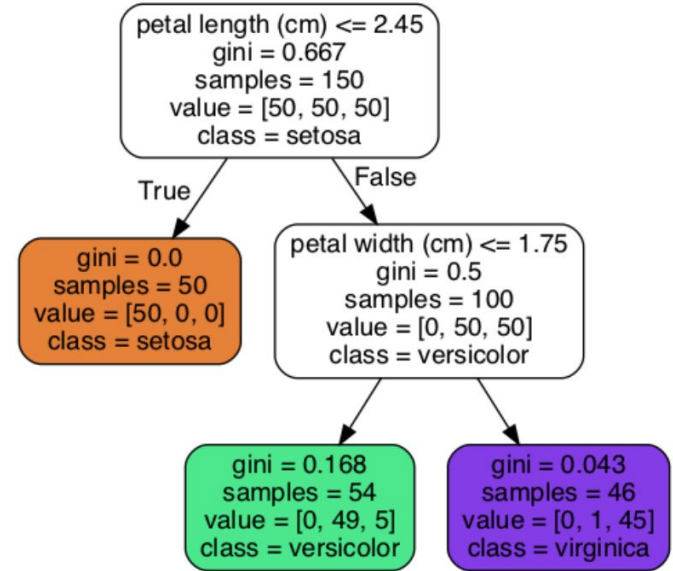
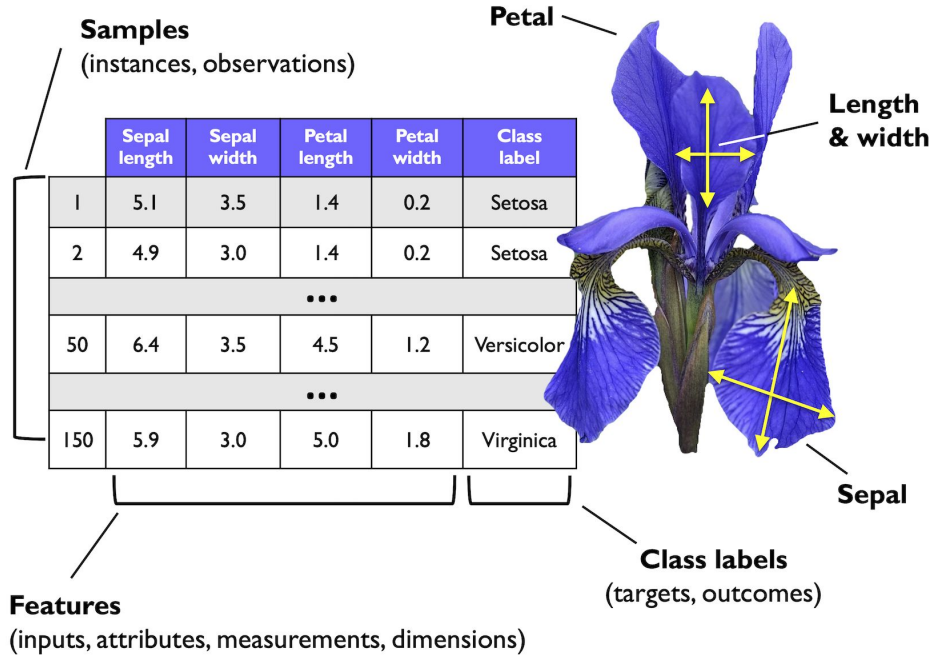
We begin by asking:

If I could only ask one question, what question would I ask?

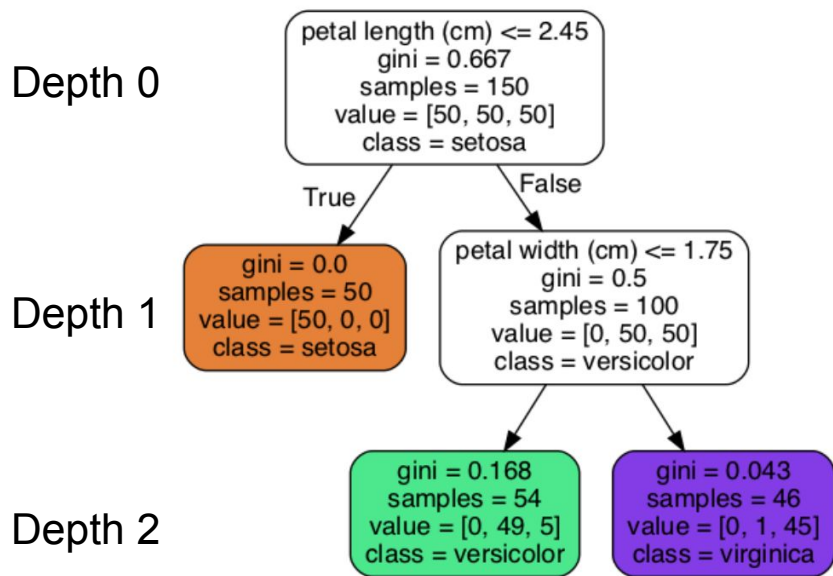
Iris Dataset: Revisited



Iris Dataset: Revisited



How to read the Tree



1. Start at the **root node** (depth 0, at the top)
2. This node asks whether the flower's petal length is smaller than 2.45 cm.
3. If it is, then you move down to the root's left **child node** (depth 1, left).
4. In this case, it is a **leaf node** (i.e., it does not have any children nodes), so it does not ask any questions.
5. Therefore, you can simply look at the predicted class for that node
6. Decision Tree predicts that your flower is an Iris-Setosa (class=setosa).

Training a Decision Tree

Using the decision algorithm, we start at the tree root and split the data on the feature that results in the largest **information gain (IG)**

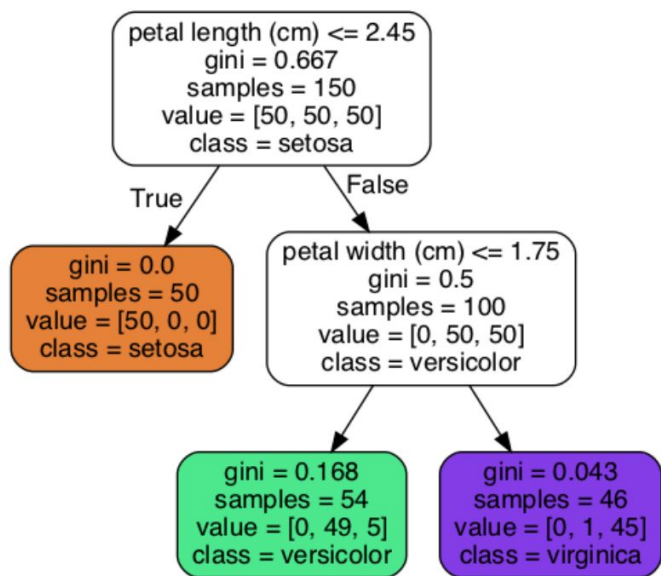
In an iterative process, we can then repeat this splitting procedure at each child node **until the leaves are pure**. This means that the training examples at each node all belong to the same class.

In practice, this can result in a very deep tree with many nodes, which can easily lead to overfitting. We typically **prune** the tree by setting a limit for the maximum depth of the tree.

Objective Function: Maximizing IG

Information gain (IG)

$$IG(D_p, f) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right})$$



f: Feature to perform the split

D_p, D_{left}, D_{right}: Dataset of the parent node, and left and right child nodes

N_p, N_{left}, N_{right}: Number of training examples in the parent node, and left and right child nodes

I: Impurity measure (more on that later)

Impurity Measures I(D)

Entropy I_H :
$$I_H(t) = - \sum_{i=1}^c p(i|t) \log_2 p(i|t)$$

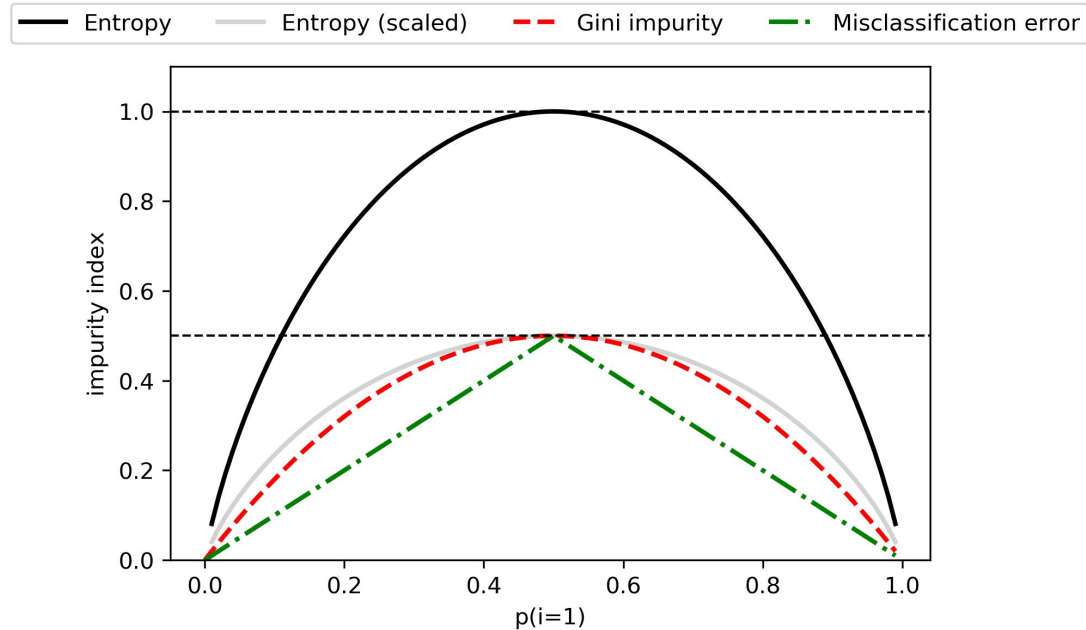
Gini Impurity I_G :
$$I_G(t) = \sum_{i=1}^c p(i|t) (1 - p(i|t)) = 1 - \sum_{i=1}^c p(i|t)^2$$

$p(i|t)$: proportion of the examples that belong to class i for a particular node, t

Impurity = 0 if all examples at a node belong to the same class,

Impurity is maximal for a uniform class distribution.

Impurity Measures $I(D)$

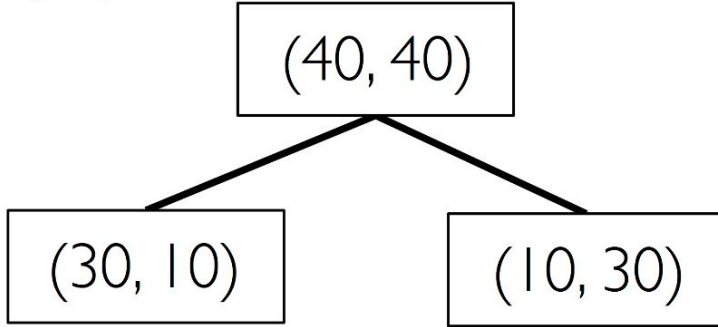


For **example**, in a binary class setting, Impurity = 0 if $p(i=1|t) = 1$ or $p(i=0|t) = 0$.

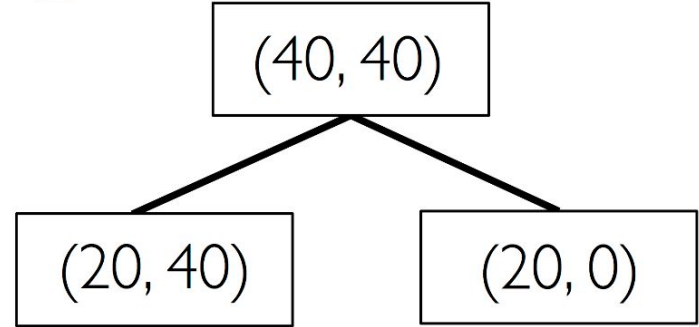
$I_H = 1$ or $I_G = 0.5$ with $p(i=1|t) = 0.5$ and $p(i=0|t) = 0.5$

Example

A



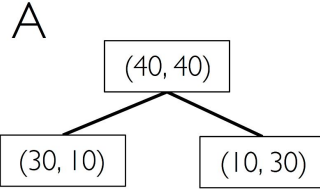
B



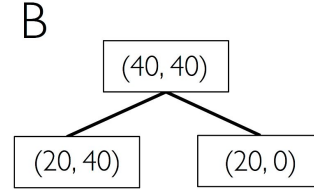
Which one do you think is better splitting?

Let's see what the impurity measures say

Example

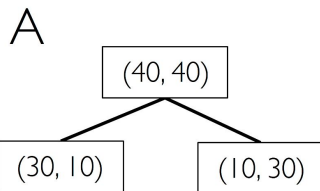


Gini impurity



Entropy

Example



Gini impurity

$$I_G(D_p) = 1 - (0.5^2 + 0.5^2) = 0.5$$

$$A: I_G(D_{left}) = 1 - \left(\left(\frac{3}{4} \right)^2 + \left(\frac{1}{4} \right)^2 \right) = \frac{3}{8} = 0.375$$

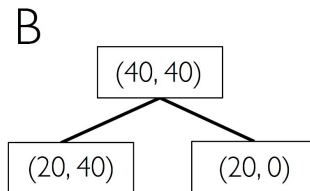
$$A: I_G(D_{right}) = 1 - \left(\left(\frac{1}{4} \right)^2 + \left(\frac{3}{4} \right)^2 \right) = \frac{3}{8} = 0.375$$

$$A: IG_G = 0.5 - \frac{4}{8} 0.375 - \frac{4}{8} 0.375 = 0.125$$

$$B: I_G(D_{left}) = 1 - \left(\left(\frac{2}{6} \right)^2 + \left(\frac{4}{6} \right)^2 \right) = \frac{4}{9} = 0.\bar{4}$$

$$B: I_G(D_{right}) = 1 - (1^2 + 0^2) = 0$$

$$B: IG_G = 0.5 - \frac{6}{8} 0.\bar{4} - 0 = 0.1\bar{6}$$



Entropy

$$I_H(D_p) = -(0.5 \log_2(0.5) + 0.5 \log_2(0.5)) = 1$$

$$A: I_H(D_{left}) = -\left(\frac{3}{4} \log_2 \left(\frac{3}{4} \right) + \frac{1}{4} \log_2 \left(\frac{1}{4} \right) \right) = 0.81$$

$$A: I_H(D_{right}) = -\left(\frac{1}{4} \log_2 \left(\frac{1}{4} \right) + \frac{3}{4} \log_2 \left(\frac{3}{4} \right) \right) = 0.81$$

$$A: IG_H = 1 - \frac{4}{8} 0.81 - \frac{4}{8} 0.81 = 0.19$$

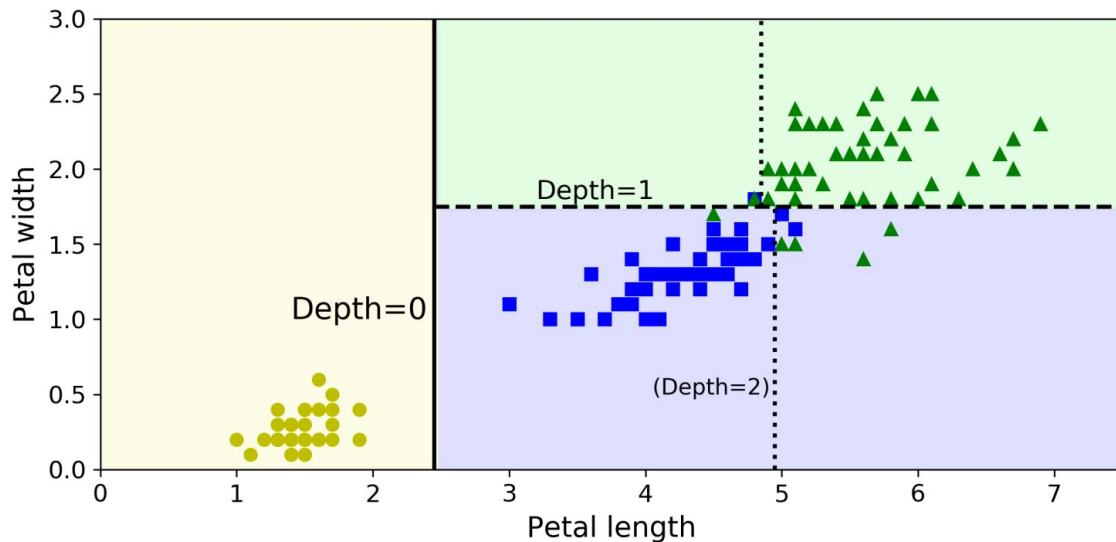
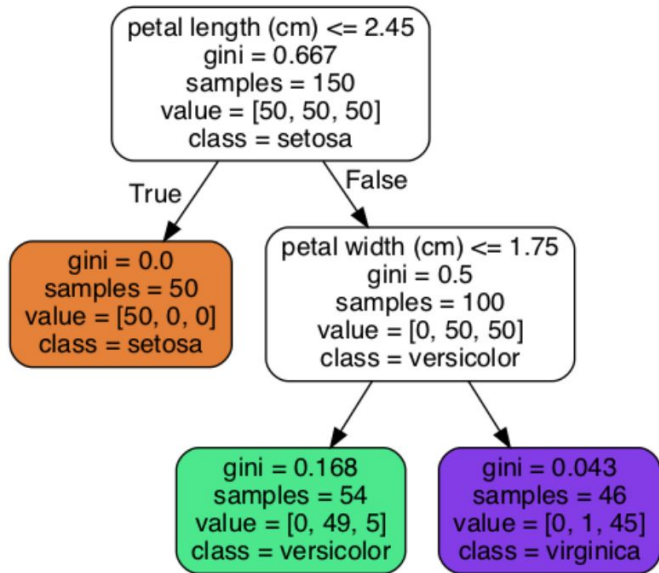
$$B: I_H(D_{left}) = -\left(\frac{2}{6} \log_2 \left(\frac{2}{6} \right) + \frac{4}{6} \log_2 \left(\frac{4}{6} \right) \right) = 0.92$$

$$B: I_H(D_{right}) = 0$$

$$B: IG_H = 1 - \frac{6}{8} 0.92 - 0 = 0.31$$

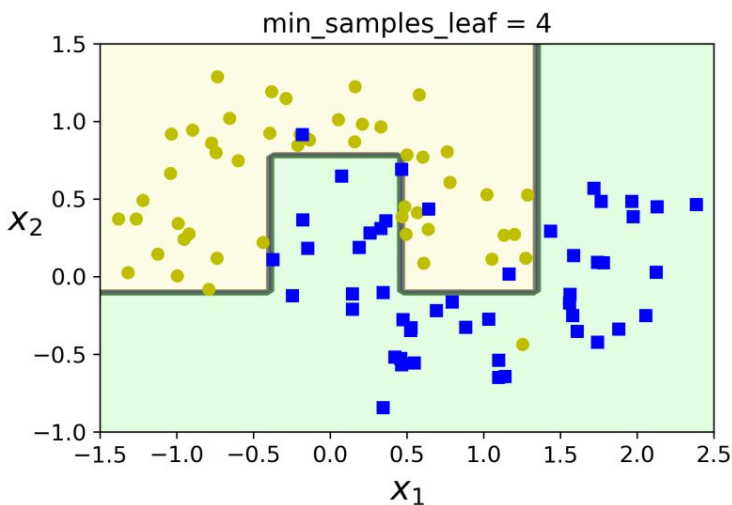
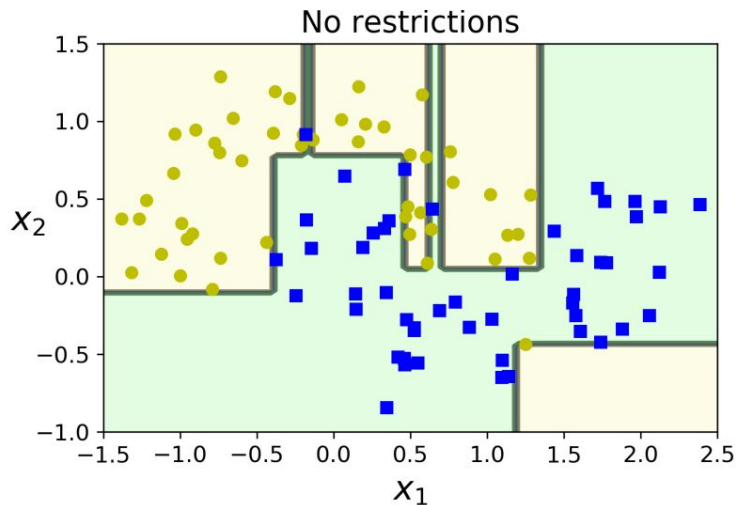
Coming Back to Iris Dataset

from `sklearn.tree` import `DecisionTreeClassifier`



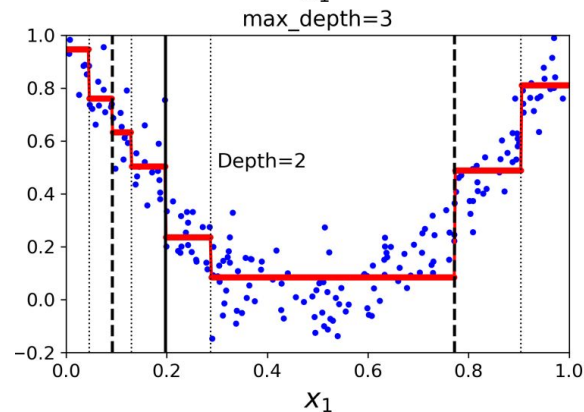
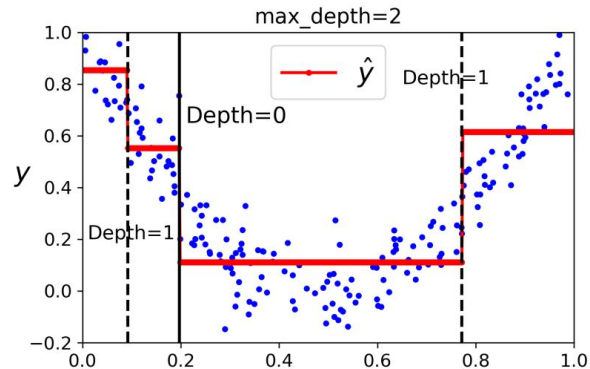
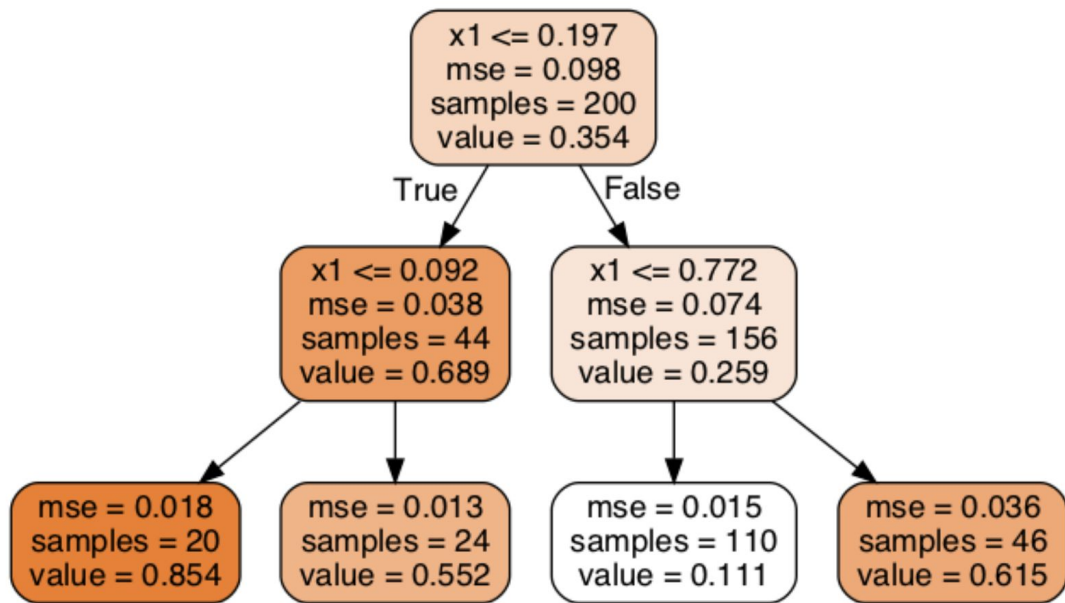
Regularization

1. **max_depth**: As we discussed, limits the depth of the tree
 2. **min_samples_split**: minimum number of samples a node must have before it can be split
 3. **min_samples_leaf**: minimum number of samples a leaf node must have
- etc...



Regression with Decision Tree

Instead of maximising Information Gain, minimize MSE



A note on interpretation

Model Interpretation: White Box Versus Black Box

As you can see Decision Trees are fairly intuitive and their decisions are easy to interpret. Such models are often called *white box models*. In contrast, as we will see, Random Forests or neural networks are generally considered *black box models*. They make great predictions, and you can easily check the calculations that they performed to make these predictions; nevertheless, it is usually hard to explain in simple terms why the predictions were made. For example, if a neural network says that a particular person appears on a picture, it is hard to know what actually contributed to this prediction: did the model recognize that person's eyes? Her mouth? Her nose? Her shoes? Or even the couch that she was sitting on? Conversely, Decision Trees provide nice and simple classification rules that can even be applied manually if need be (e.g., for flower classification).

Example of Decision Tree

Building Block-Based Binding Predictions for DNA-Encoded Libraries

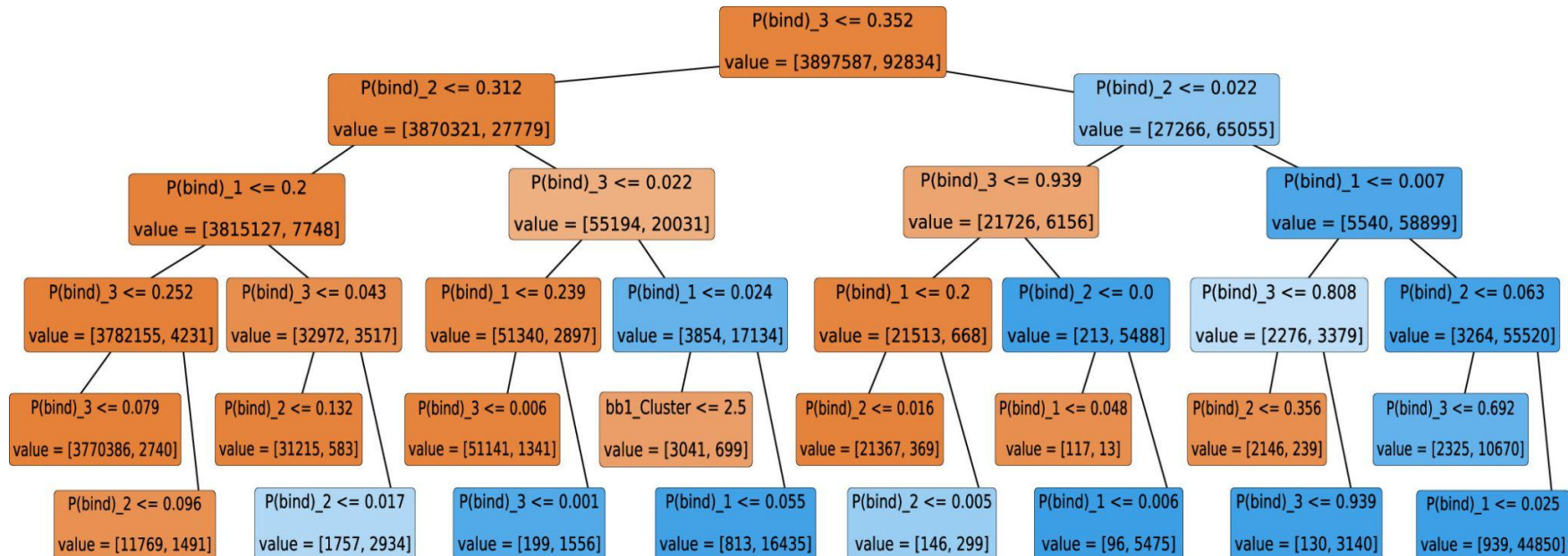
Chris Zhang, Mary Pitman, Anjali Dixit, Sumudu Leelananda, Henri Palacci, Meghan Lawler, Svetlana Belyanskaya, LaShadric Grady, Joe Franklin, Nicolas Tilmans, and David L. Mobley*



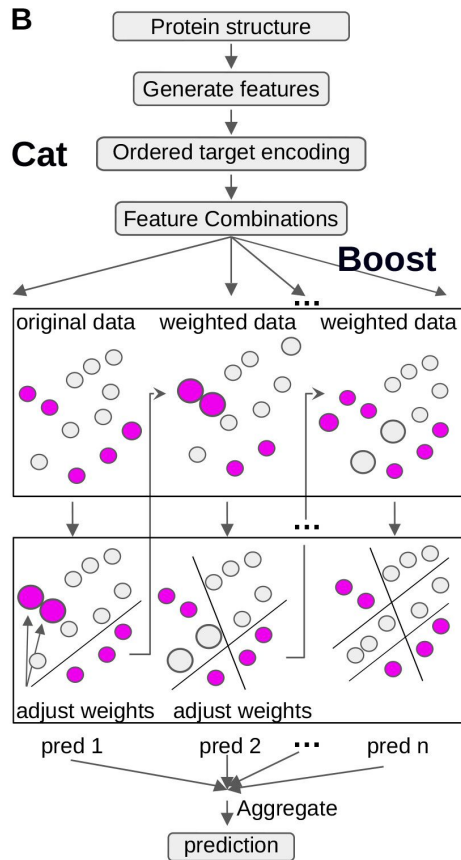
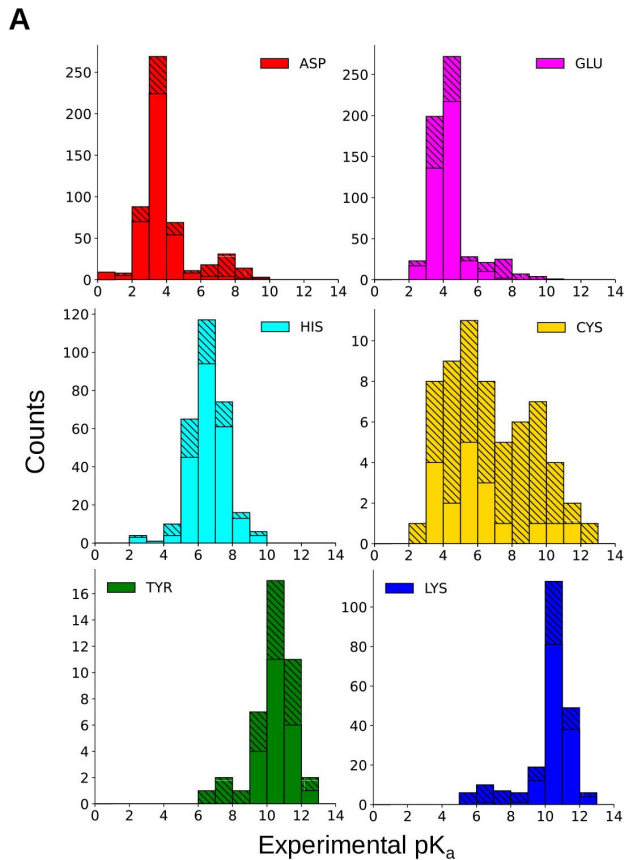
Cite This: *J. Chem. Inf. Model.* 2023, 63, 5120–5132



[Read Online](#)



Example of Decision Tree



KaMLs for Predicting Protein pK_a Values and Ionization States: Are Trees All You Need?

Mingzhe Shen, Daniel Kortzak,
Simon Ambrozak, Shubham
Bhatnagar, Ian Buchanan,
Ruibin Liu, and Jana Shen
*Journal of Chemical Theory
and Computation* Article ASAP
DOI: 10.1021/acs.jctc.4c01602

Jupyter Notebook Tutorial

Ensemble Learning

Groups of people can often make better decisions than individuals.

The same is true in machine learning.

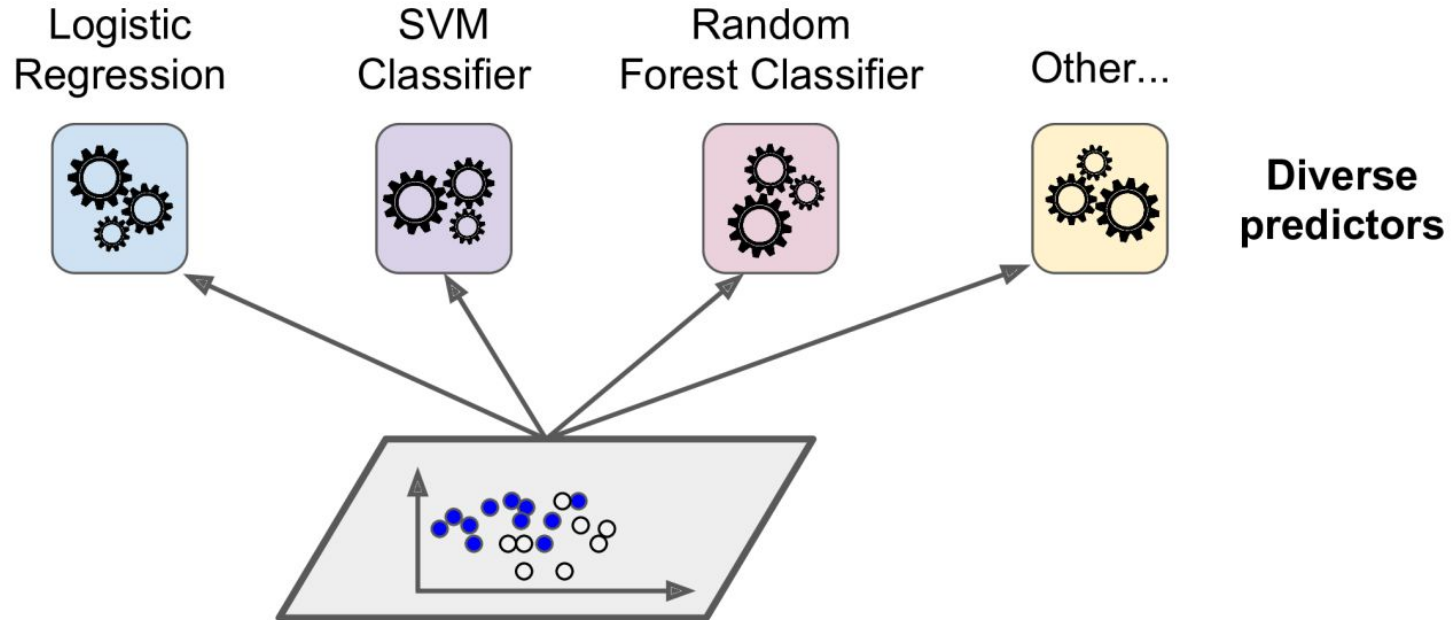
Ensemble methods are learning models that achieve performance by combining the opinions of multiple learners.

Advantages:

1. Using simpler learners (e.g decision tree) and still achieve great performance.
2. Ensembles are inherently parallel, efficient training

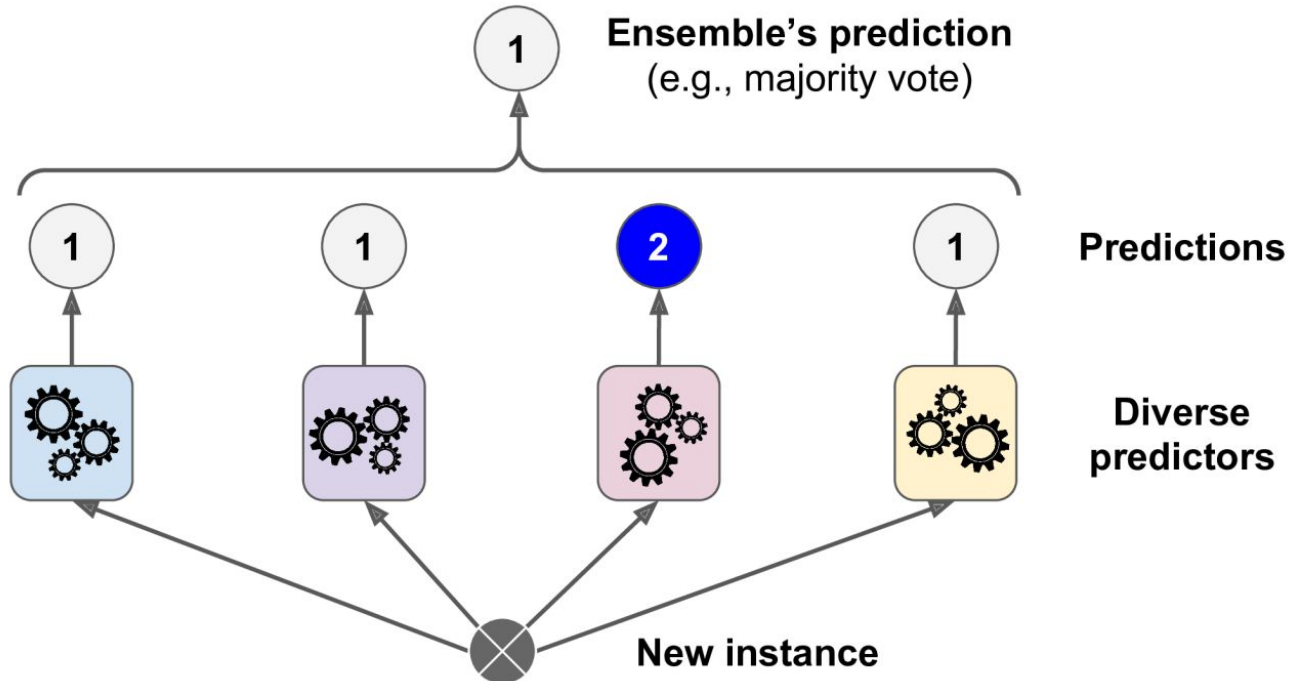
Voting Classifier

Train different classifiers from the same data ...



Voting Classifier

Then take a majority vote: select the most frequent one



Voting Classifier

Why does it work?

Voting Classifier

Why does it work?

Assume you have 3 independent classifiers, each 60% accurate

The probability that at least two of them gives the right answer is

$$\begin{aligned} &0.6*0.6*0.4 + 0.6*0.4*0.6 + 0.4*0.6*0.6 + 0.6*0.6*0.6 \\ &= 0.648 \\ &= 64.8\% \end{aligned}$$

There is a higher chance of getting the correct answer if we do voting

This effect is magnified with a large number of (100+) classifiers.

This is the main principle of **ensemble learning**

Advantage of Ensemble Learning

Transformer (used in
e.g. ChatGPT)

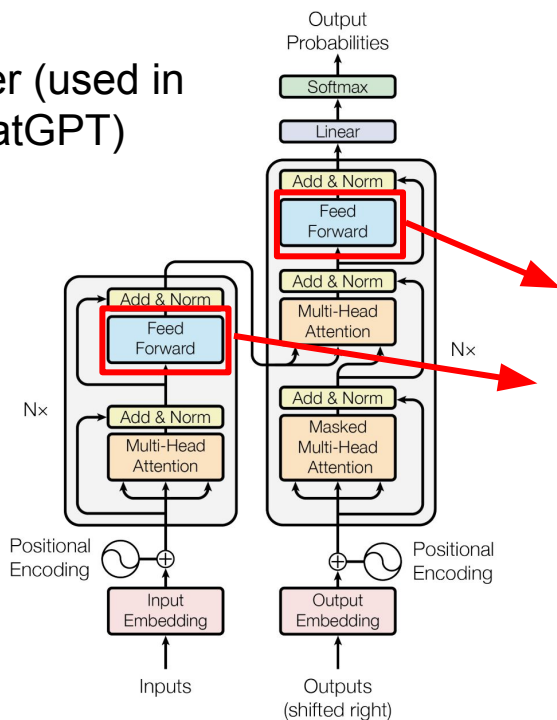
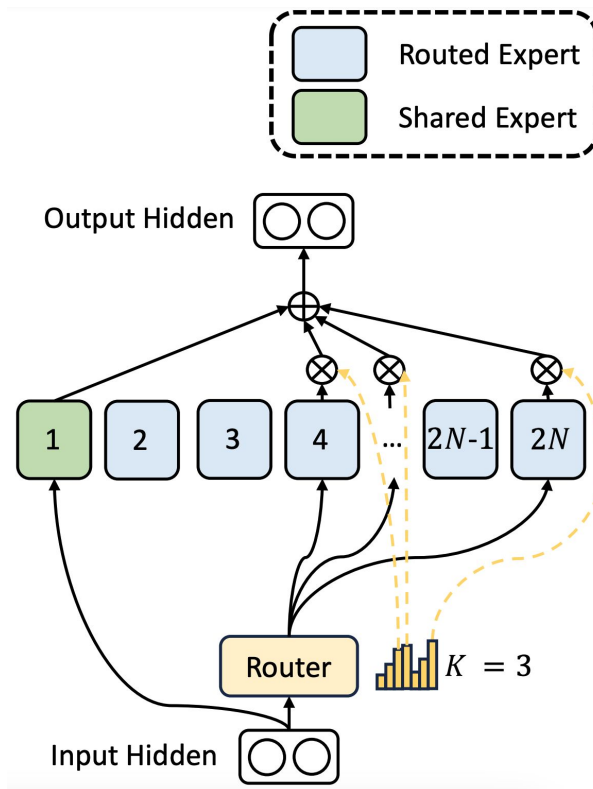


Figure 1: The Transformer - model architecture.



DeepSeekMoE:
Towards Ultimate
Expert Specialization
in Mixture-of-Experts
Language Models.

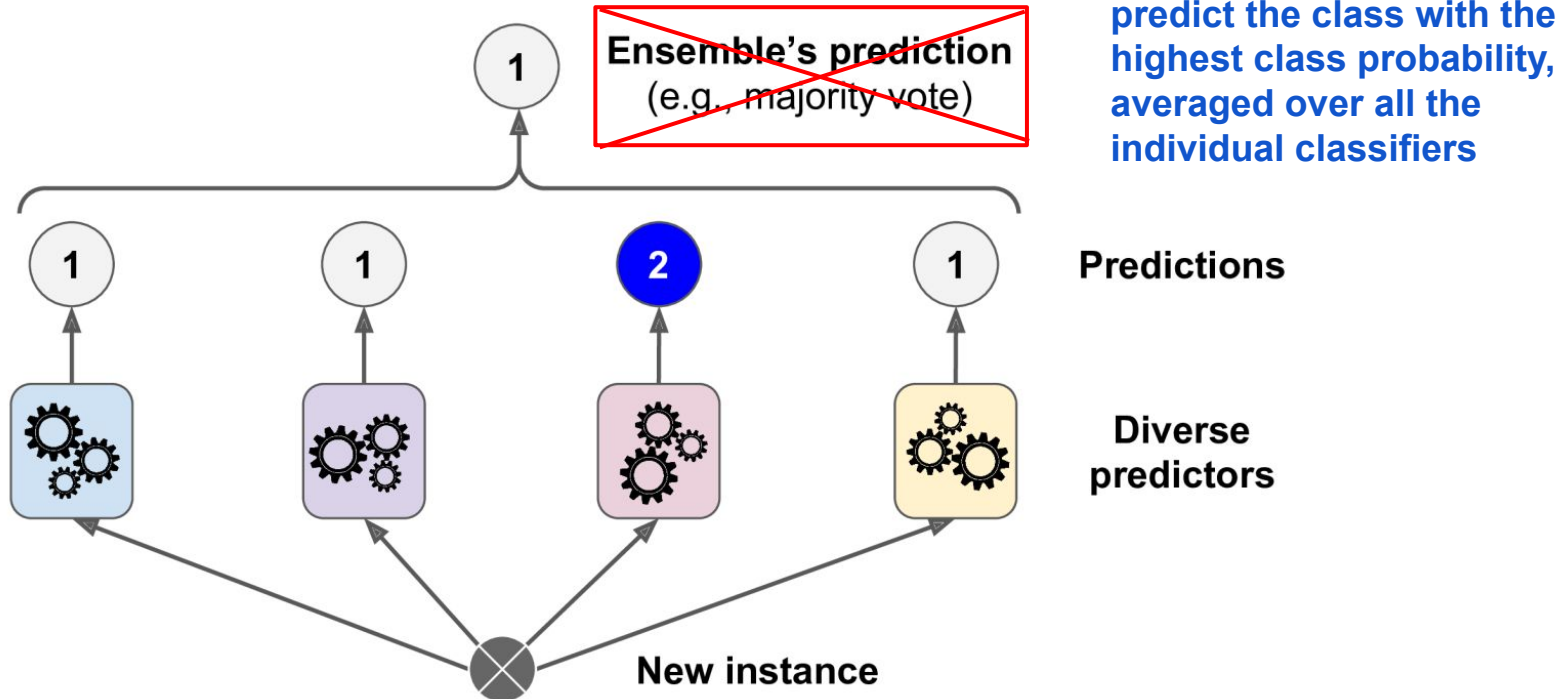
Dai et al.

<https://arxiv.org/abs/2401.06066>

MoE =
Mixture-of-Experts
(Univ. Sheffield, UK)

Soft Voting Classifier

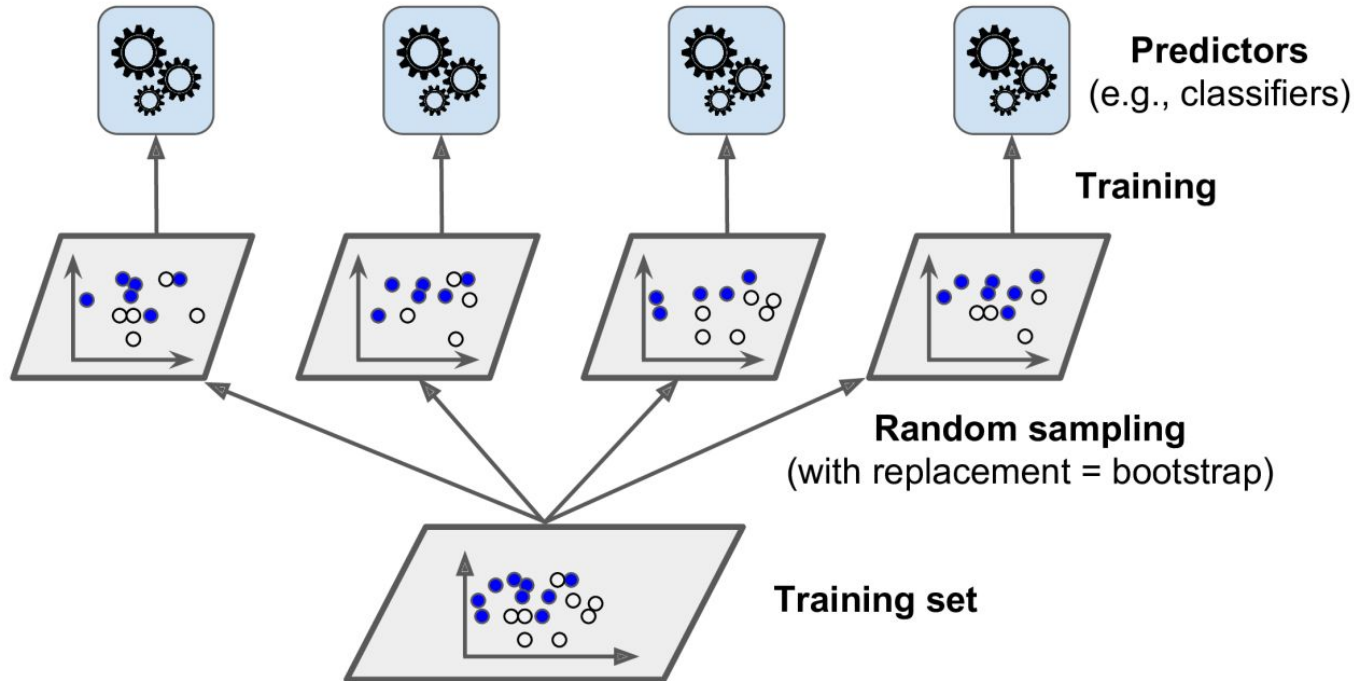
Instead of majority take average of all. Often more accurate



Another approach: Bagging/Pasting

Use the same training algorithm for every predictor, but to train them on different random subsets of the training set

Training with replacement = bagging, without replacement = pasting



Another approach: Bagging/Pasting

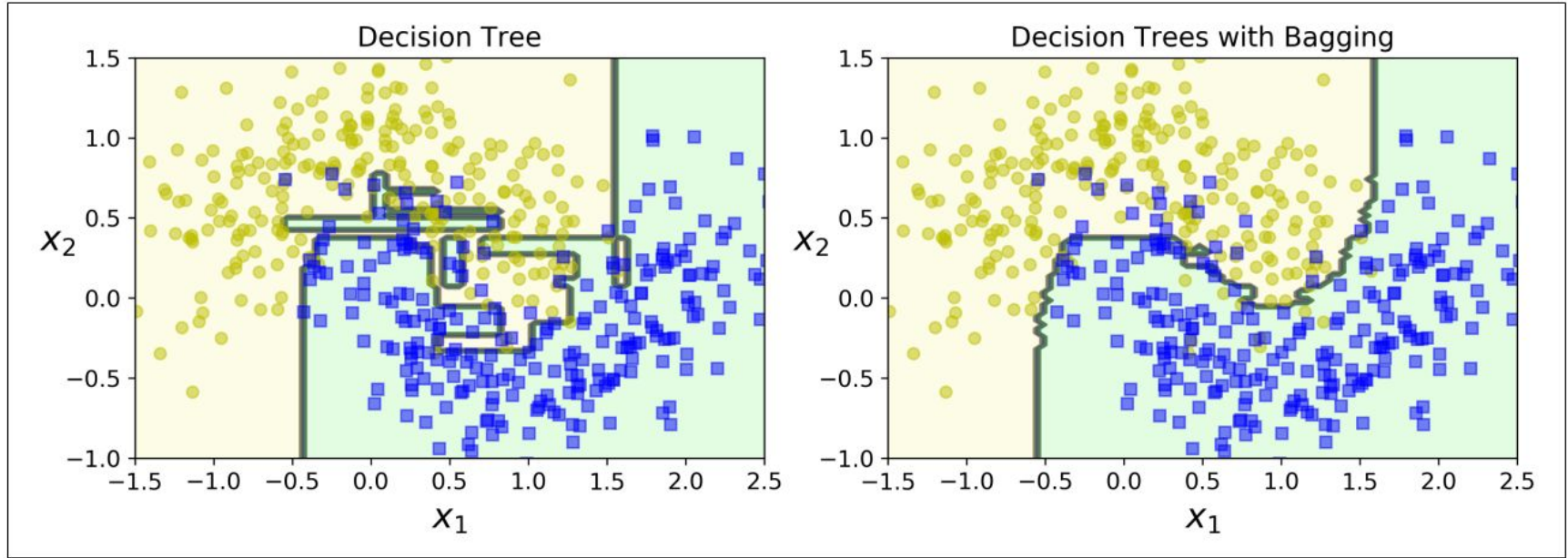


Figure 7-5. A single Decision Tree versus a bagging ensemble of 500 trees

Random Forest

One of the most commonly used decision tree based methods in Science and Engineering

Random Forest is an ensemble of Decision Trees, generally trained via the bagging method (or sometimes pasting), typically with a limit on the sample size used per decision tree

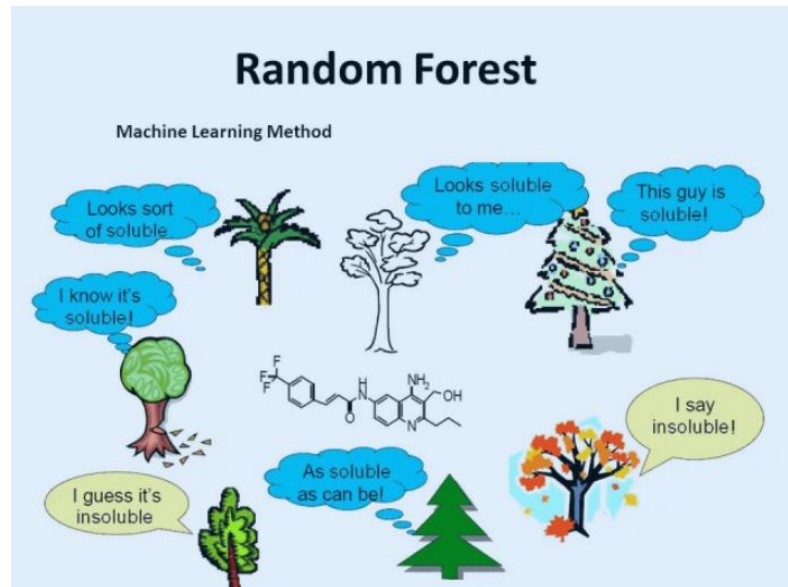
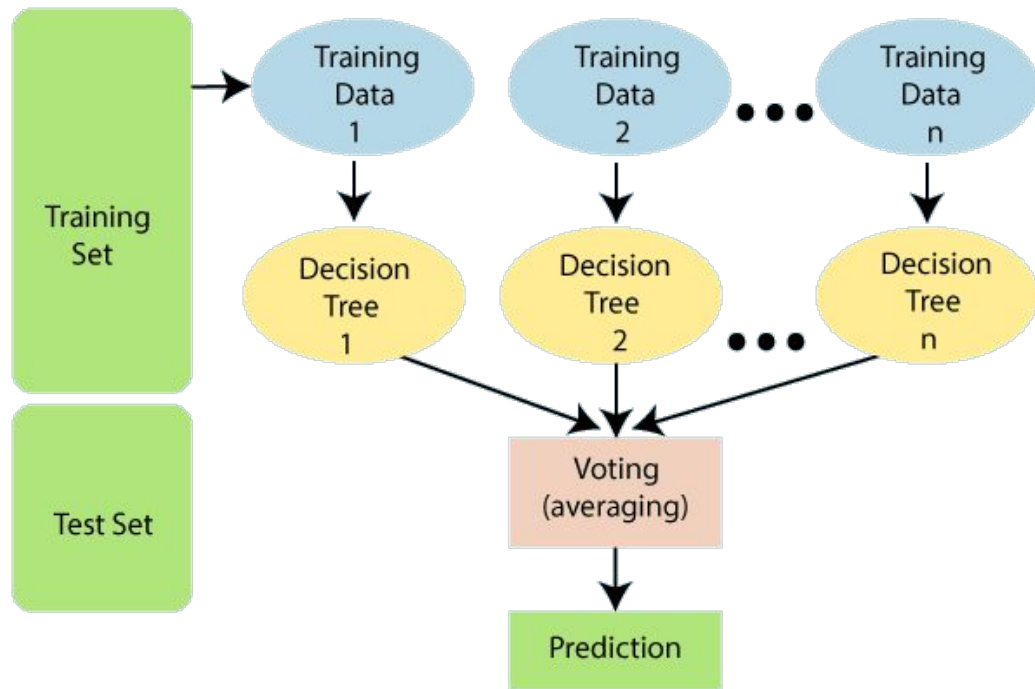


Figure by Harshini Bhat

Random Forest



Random Sampling: For each decision tree in the ensemble, a random subset of the training data is selected

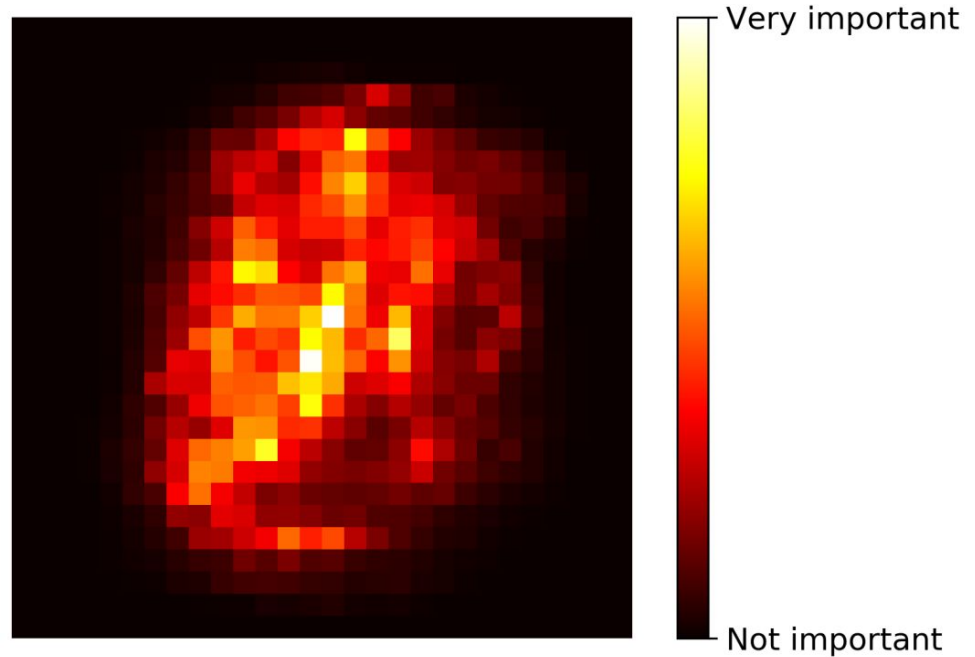
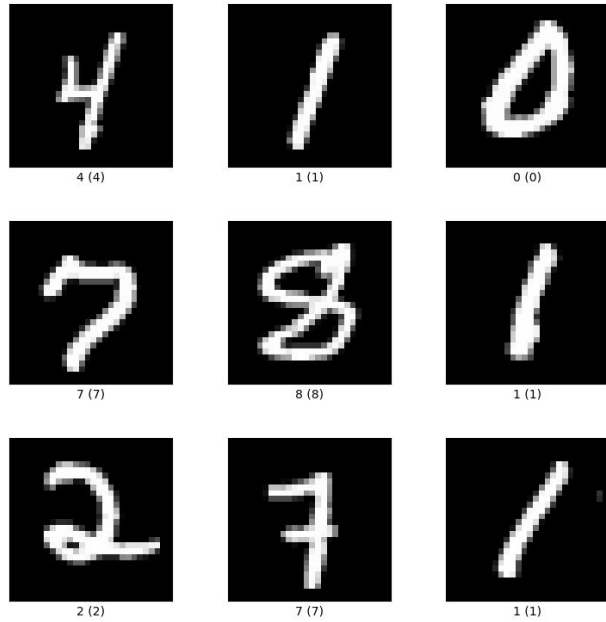
Random Feature Selection: At each split point of a decision tree, a random subset of features is considered

Validation is performed on the final prediction of the forest

Figure by Harshini Bhat

Random Forest: Interpretability via Feature Importance

Feature importance: how much the tree nodes that use that feature reduce impurity on average (across all trees in the forest)



MNIST dataset of handwritten digits

Example: Atmospheric Chemistry

Numerical model

Emulator

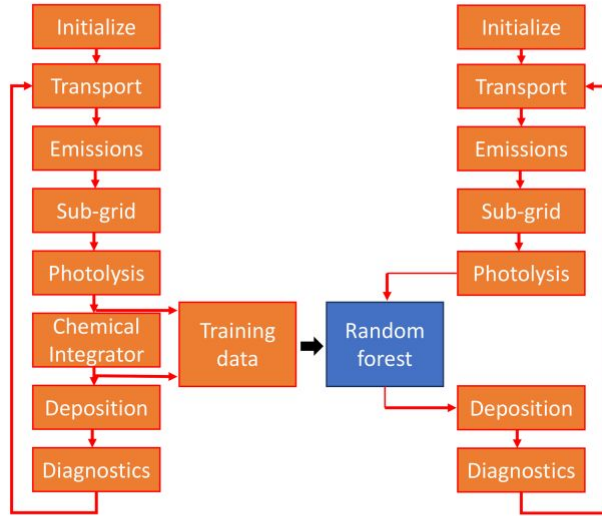
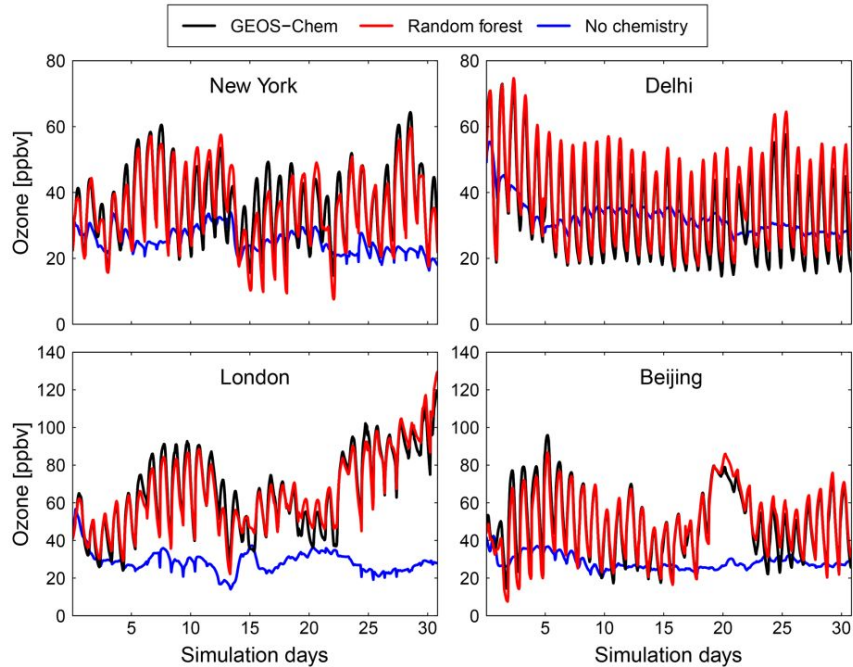
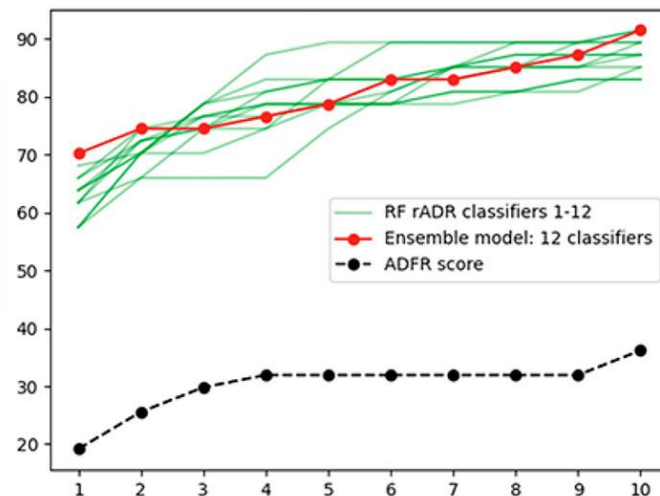
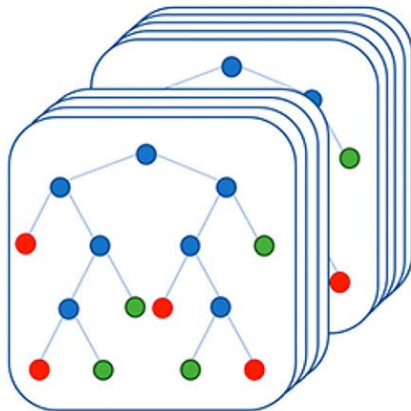
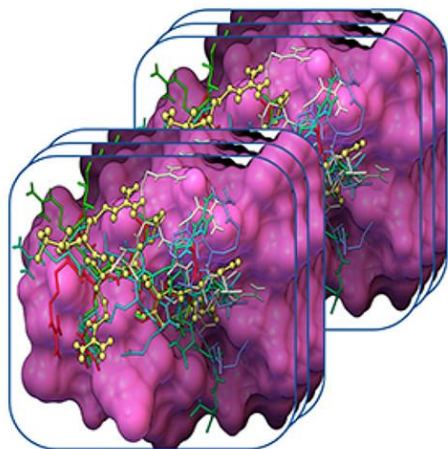


Figure 1. Schematic overview of the use of a random forest regression algorithm as an alternative to the chemistry solver. The original numerical model (GEOS-Chem) sequentially solves the operations relevant to atmospheric chemistry, with the chemical integrator being the computationally most expensive step (left side). Using training data produced from the full model, we generate a machine learning emulator that can then be used instead of the chemical integrator (right side). All other model processes are the same as in the original model.



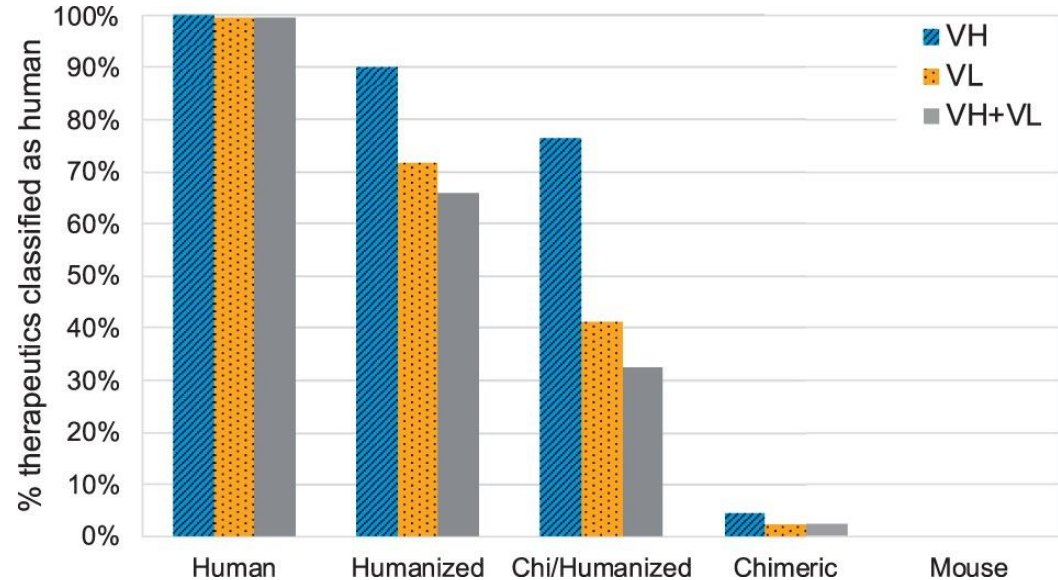
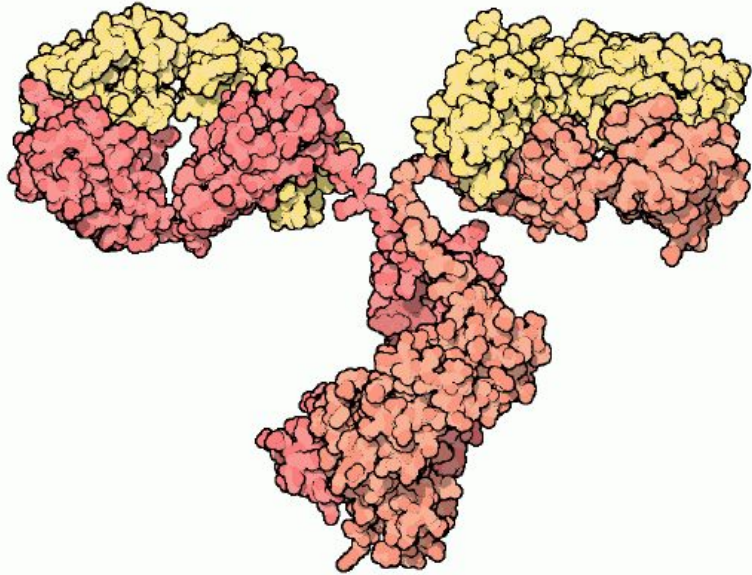
Keller & Evans. Geosci. Model Dev., 12,
1209–1225, 2019

Example: Molecular docking of Peptide drugs



Michel F. Sanner, Leonard Dieguez, Stefano Forli, and Ewa Lis, Journal of Chemical Information and Modeling **2021** 61 (6), 3074-3090. DOI: 10.1021/acs.jcim.1c00573

Example: Antibody design



Humanization of antibodies using a machine learning approach on large-scale repertoire data

Claire Marks, Alissa M Hummer, Mark Chin, Charlotte M Deane

Bioinformatics, Volume 37, Issue 22, November 2021, Pages 4041–4047,

<https://doi.org/10.1093/bioinformatics/btab434>

Boosting

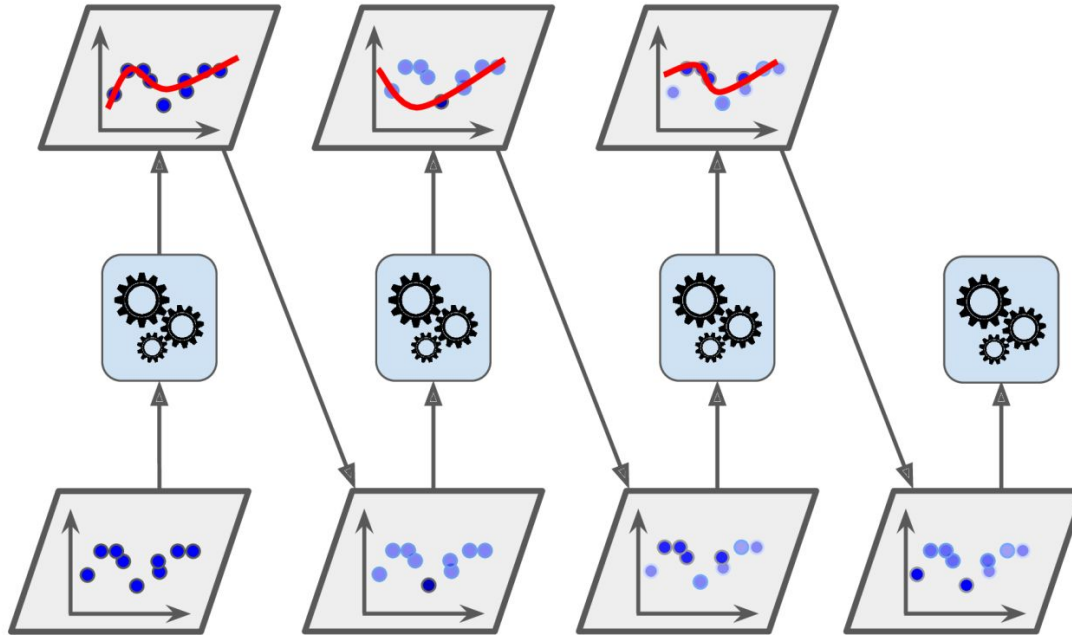
Train predictors sequentially, (not parallelly) each trying to correct its predecessor

Two most common types

1. AdaBoost
2. Gradient Boosting (used in science and engineering)

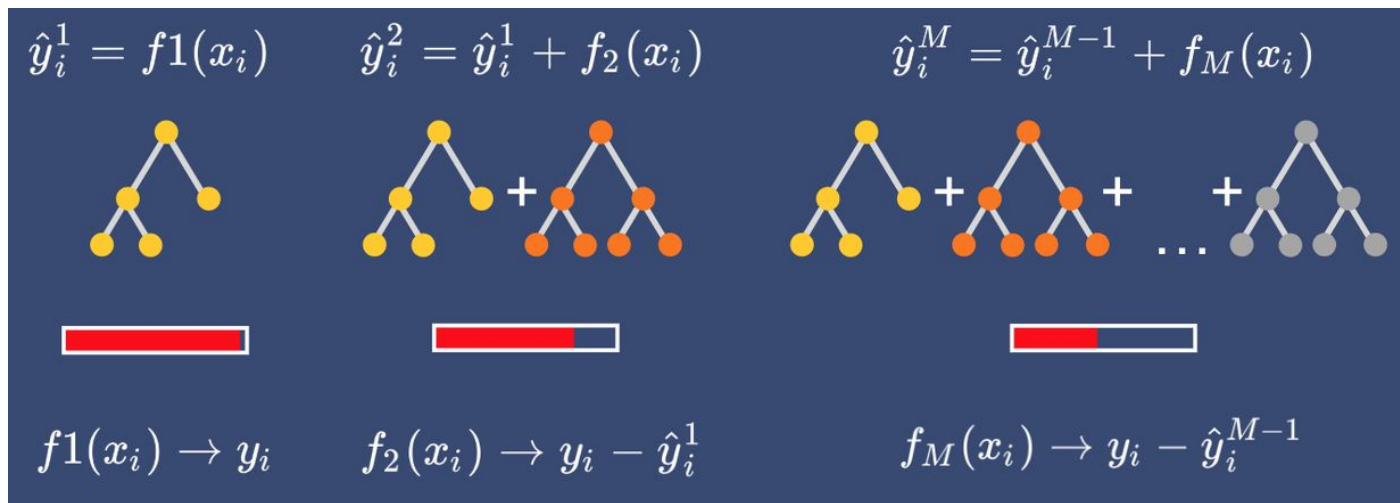
AdaBoost

The relative weight of misclassified training instances is then increased. A second classifier is trained using the updated weights, and so on.



Gradient Boosting

Instead of tweaking the instance weights at every iteration like AdaBoost does, **Gradient Boosting** tries to fit the new predictor to the **residual errors** made by the previous predictor



Gradient Boosting (code example)

```
from sklearn.tree import DecisionTreeRegressor
```

```
tree_reg1 = DecisionTreeRegressor(max_depth=2)  
tree_reg1.fit(X, y)
```

Now train a second DecisionTreeRegressor on the residual errors made by the first predictor:

```
y2 = y - tree_reg1.predict(X)  
tree_reg2 = DecisionTreeRegressor(max_depth=2)  
tree_reg2.fit(X, y2)
```

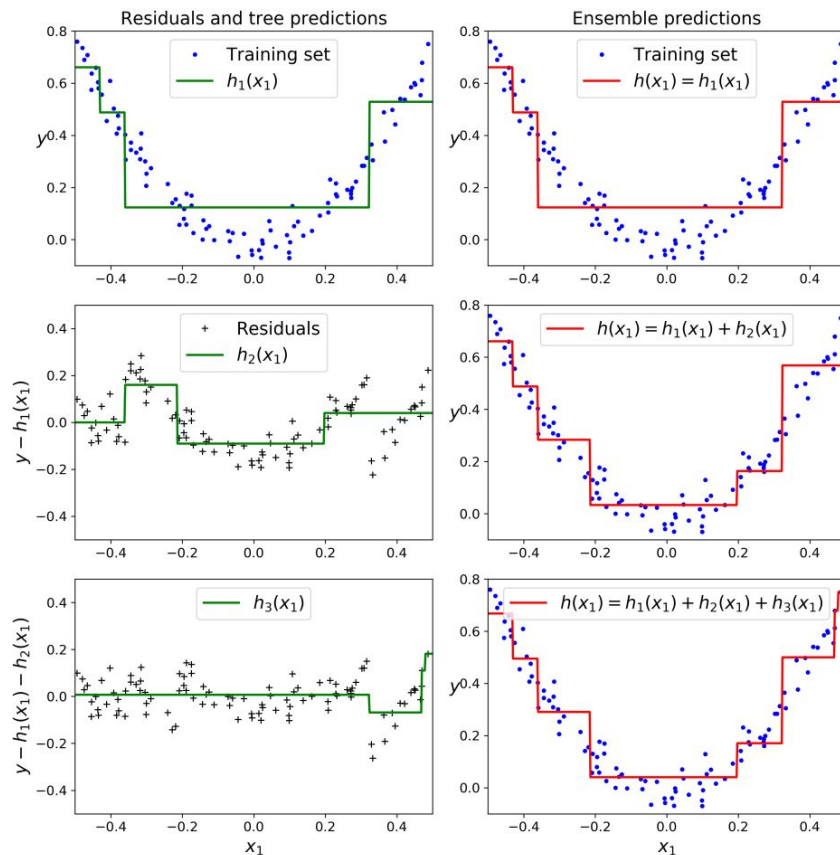
Then we train a third regressor on the residual errors made by the second predictor:

```
y3 = y2 - tree_reg2.predict(X)  
tree_reg3 = DecisionTreeRegressor(max_depth=2)  
tree_reg3.fit(X, y3)
```

Now we have an ensemble containing three trees. It can make predictions on a new instance simply by adding up the predictions of all the trees:

```
y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
```

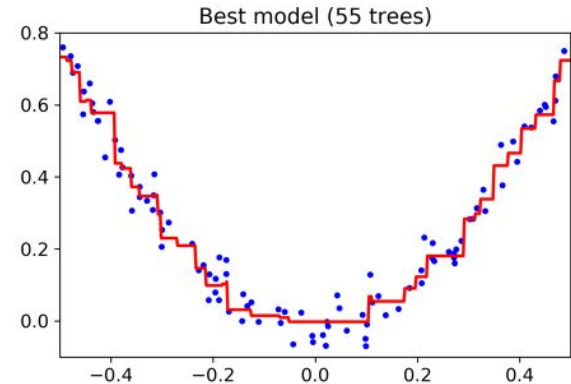
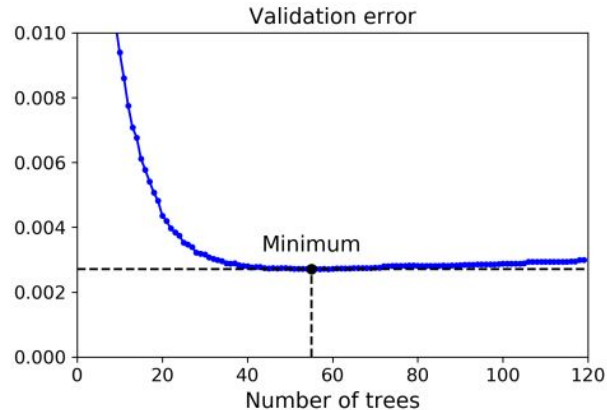
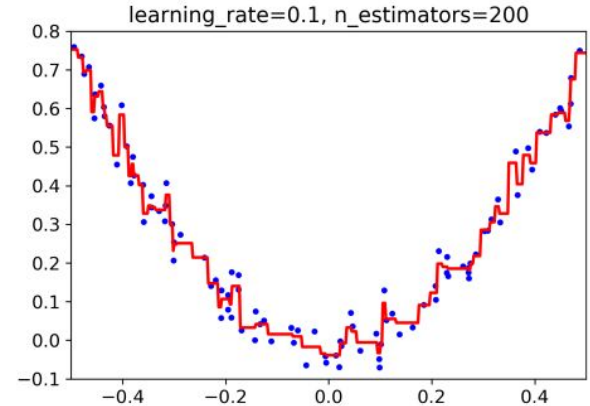
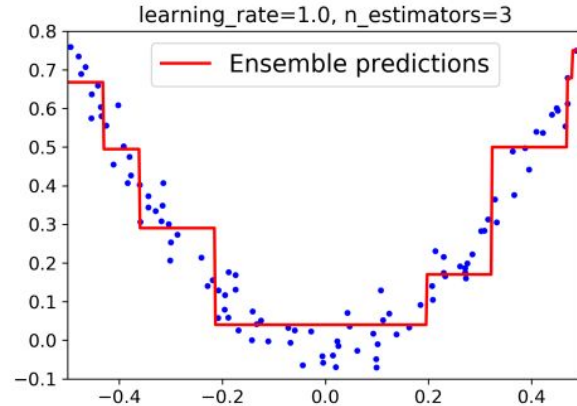
Gradient Boosting (example)



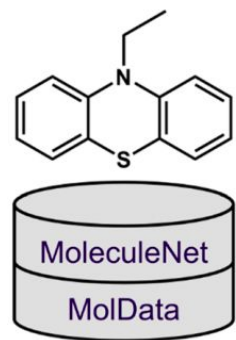
Gradient Boosting (avoid overfitting)

Too many estimators
(trees) can cause
overfitting

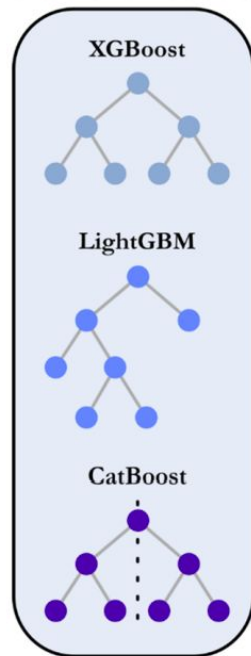
Use early stopping
regularization to
determine optimum
number of trees



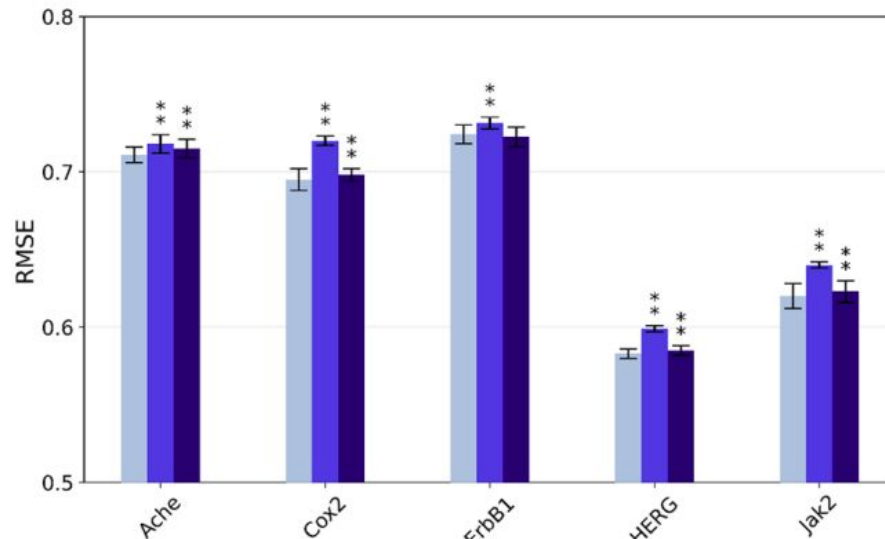
Example: molecular property prediction



Gradient Boosting



Drug-target binding affinity (IC_{50})



Boldini et al. Journal of Cheminformatics (**2023**) 15:73